

Velora MERN Architecture

System design, trust boundaries, data model and delivery structure

Version 2.0 | 10 October 2026 | Swayam Jain

01 / Architecture approach and trust boundaries

Version 2.0 - JavaScript MERN baseline. Requirements authority: VELORA_MERN_MASTER. Review protocol and progress authority: VELORA_MERN_DUCK_LOG.

Verified current repository truth: GitHub reports repository size 0 and no branches or commits. Part 1 will initialize the JavaScript MERN implementation only after inspecting and preserving any local files.

Deployable shape

Component	Responsibility
React + Vite client	Public landing and role-specific accessible application. Holds no database/provider secret.
Node + Express API	Authentication, authorization, validation, workflows, audit and provider adapters.
MongoDB + Mongoose	Tenant-scoped relational-style documents, indexes, sessions/transactions and history.
Private object-storage adapter	Encrypted document originals and authorized exports; local private adapter first.
Job/worker boundary	Later extraction, notifications, export and retention jobs with current-access/version rechecks.
Future AI adapter	Strict source-linked draft schema. No publication, prescription or unrestricted mutation authority.

The browser, uploaded files, model output and all user input are untrusted. Request path: React -> Express route -> authentication/CSRF -> validation -> server policy -> domain service/transaction -> Mongo/private storage. UI hiding never replaces server authorization.

Initial architecture choice

Use a modular monolith with clear feature/service boundaries. A separate AI service may be introduced later only when isolation or workload evidence justifies it. Begin with an internal provider interface so keys and policy remain behind the API.

02 / Repository and module structure

```
velora/  
client/  
src/app/ router, providers, protected shell  
src/features/ auth, clinic, patients, episodes, documents, plans...  
src/components/ shared viewport-safe UI  
src/lib/ API client, validation helpers  
tests/ component and browser support  
server/  
src/config/ validated environment  
src/models/ Mongoose schemas and indexes  
src/validators/ request/output schemas  
src/policies/ clinic/record/state decisions  
src/services/ workflows and transactions  
src/controllers/ request/response coordination  
src/routes/ API routing  
src/middleware/ auth, CSRF, errors, limits, headers  
src/jobs/ bounded background work  
src/providers/ storage, mail, AI adapters  
tests/ unit/integration/security  
docs/ three authorities, API contracts, part reviews  
.github/workflows/ full verification  
.env.example names/placeholders only  
.gitignore
```

Do not create empty architecture folders without a responsibility. Routes stay thin. Policies decide authorization. Services own transactions and state. Models persist facts and indexes. Controllers map validated requests to services. Background jobs call the same services and recheck current authority.

Environment boundary

MONGODB_URI, session/hash/encryption keys and provider credentials exist only in server runtime configuration. The client may receive only public values such as the API origin. .env, private uploads, keys, exports, backups, logs and test artifacts are ignored; .env.example is explicitly allowed and contains no real values.

03 / Domain model by delivery part

Part	Core entities
1	User, Clinic, ClinicMembership, MembershipRole, Invitation, Session, MFA recovery, PasswordReset, RateLimitBucket, AuditEvent
2	PatientProfile, PatientAssignment, PatientConsent, CaregiverGrant, AccessInvitation
3	RecoveryEpisode, CareTeamAssignment, EpisodeCorrection
4	EpisodeDocument plus private object metadata
5	CarePlanVersion, CarePlanItem, SourceReference, ReviewEvent
6	ExtractionJob, DraftSuggestion, SourceCitation, ModelRun, ReviewedLanguageVariant
7	MedicationSchedule, RecoveryTask, ScheduleOccurrence, ActivityLog
8	CheckInTemplateVersion/Response, ClarificationThread/Message, QueueAssignment, Notification/DeliveryAttempt
9	DataRequest, RetentionPlan/Run and expanded audit/closure history

MongoDB rules

Use immutable clinic/record relationships, compound unique and partial indexes, UTC timestamps and explicit timezones. Use Mongoose sessions/transactions on a replica-set-capable database for multi-document invariants. Use atomic revision filters so parallel state changes cannot both succeed. Avoid arbitrary query objects and operator injection; use allowlisted projections and filters.

Never auto-merge people from names, phone or email. Never trust a client-supplied role, clinic or ownership relationship. Derive clinic from the target record and verify linked membership/ownership/assignment/grant.

04 / Access policy, authentication and APIs

Authentication

Passwords use Argon2id. Sessions are opaque random values, stored hashed server-side and delivered in secure httpOnly cookies with expiry, rotation and revocation. Staff MFA is required. Reset, invitation and recovery values are expiring, single-use and stored only as hashes. Cookie mutations require trusted Origin plus CSRF protection.

Authorization decision

Authenticate active identity -> resolve target and clinic server-side -> verify membership/owner -> verify role plus record assignment or caregiver scope -> verify workflow state/version -> apply limits -> execute service. Default deny. Apply the same policy to lists, counts, downloads, exports and background work.

Status	Safe meaning
400/422	Malformed or semantically invalid request without reflecting sensitive input.
401	Authentication/MFA/session required.
403	Known operation denied only where disclosure is safe.
404	Scoped target unavailable; avoid cross-tenant existence leak.
409	State, duplicate, stale revision or incomplete approval conflict.
413/415	Upload size/type rejected.
429	Rate/resource limit.
500/503	Generic internal/dependency unavailable; no stack/secret/provider detail.

Use a consistent JSON envelope such as { error: { code, message } }. Successful resources expose only documented fields; never token hashes, internal storage keys, encryption material or unrelated PII.

05 / State machines and transaction boundaries

Aggregate	State and invariant
Episode	draft -> active after approved plan -> completed -> archived. Closure is explicit and operational.
Document	uploaded/validating -> ready, rejected, quarantined or failed. Ready is technical only.
Care plan	draft -> in_review / needs_clarification -> approved; replacement approval supersedes the old current version atomically.
Clarification	open -> assigned -> answered -> closed; reopen is explicit and audited.
Notification	queued -> delivered or failed; delivery and acknowledgement remain separate.
Privacy request	received -> verified -> assigned/reviewed -> completed/denied with retained evidence.

Critical transactions

- Patient/episode/team creation and revision writes use unique indexes, transactions and atomic expected-revision checks.
- Approval validates completeness and reviewer eligibility, approves one version, supersedes the prior current version, updates the episode pointer and audits in one transaction.
- Document metadata and filesystem/object storage cannot form one distributed transaction; reconciliation and backup/restore cover orphan/missing/corrupt cases.
- Version updates reconcile only future schedules/reminders; historical logs remain immutable.
- Closure stops allowed future work and preserves read-only history; it never asserts medical recovery.

06 / Documents, AI and background reliability

Private documents

Accept bounded PDF/JPEG/PNG only. Check filename, declared type, extension and magic bytes; use isolated resource-limited parsing. Reject active/encrypted/malformed/excessive PDFs and image pixel bombs. Encrypt immutable originals with opaque keys outside the web/repository root. Fresh authorization and integrity checks precede every download.

AI boundary

No provider is selected. Before any SDK/network use, disclose provider, purpose, model, key/env, backend location, data sent, retention/region and cost. Provider output must match a bounded schema and cite supplied pages/text. Prompt injection in documents is data, not authority. Manual drafting remains available. Applying a suggestion is an actor-attributed draft edit and never approval.

Jobs and reminders

Use idempotency/deduplication, bounded retry and visible failure. Before work and before applying/delivering, recheck current session-independent authorization, episode state, plan version and grant. No failed job becomes a silent success. In-app notification comes first; external messaging requires its own disclosure and data review.

07 / UI, verification and delivery discipline

Authenticated UI

Application pages fit one viewport: wizards, server pagination, tabs/routes, switchable charts and one primary mobile panel. Never cut content with overflow:hidden. Clear sensitive client state after denied refresh, logout, revocation or scope change. Render user/document/model content as text.

Verification layers

Layer	Evidence
Unit	Validators, policies, state machines and deterministic helpers.
Mongo integration	Indexes, replica-set transactions, concurrency, tenant isolation and idempotency.
API/security	Authentication, CSRF, CORS, safe errors, rate limits, NoSQL injection and redaction.
Browser	Real flows and denied refresh at six viewport sizes with reachable controls and no page scroll.
Supply chain	Pinned lockfiles, runtime audit, compatibility review and unused-package removal.
Secrets	Current tree and full reachable-history scan.
AI evaluation	Field/source/missing/conflict performance measured separately from software tests.

After coherent work or about 300 meaningful LOC: run relevant tests, inspect working and staged diff, commit with a meaningful message and push when authorized. Aim for 50+ real commits across ten parts without padding. Each part ends with full relevant regression, Security-25 matrix, Duck review, docs and limitations.